ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE PRÁCTICA N.º2

Clasificación de variables y población vs. muestra

Tópico 1 del PEA · Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante(s)	Tiupul Jhon — usuario de GitHub: jhont-ml
Docente	Ing. Christian Suárez, M.Sc.
Período académico	2026-B
Fecha de entrega	23/10/2026
Repositorio (fork)	https://github.com/jhont-ml/IDSD422-lab-muestreo-poblacion.git

Resumen ejecutivo

Una muestra de 20 clientes sí puede servir para conocer el ingreso promedio de la base, pero solo si se elige por sorteo. En el laboratorio comparé tres formas de tomar esas 20 personas contra el valor real de la población, que es de 1330.72 USD. Tomar las primeras 20 filas del archivo dio 1348.50 USD, un número que parece bueno, pero esa muestra estaba formada únicamente por clientes de Quito. El sorteo simple dio 1336.00 USD y el sorteo por ciudades 1287.50 USD. Al repetir el sorteo simple con cinco semillas, las medias se movieron entre 1235.00 y 1491.50 USD, con una desviación de unos 100 USD. Antes de esto revisé las ocho columnas de la base y encontré que dos de ellas, edad y satisfaccion, engañan si uno se guía solo por el tipo de dato que asigna el programa.

1. Introducción

Imaginemos que el área comercial de RetailAndina S.A. presenta un reporte que dice la satisfacción promedio de nuestros clientes es 3.6 y que, además, calcula el ingreso promedio con los primeros registros que encontró en el archivo. Las dos cifras se pueden calcular sin que el programa marque ningún error, pero ninguna de las dos es confiable. Este laboratorio parte justamente de ese riesgo. La gerencia necesitaba saber qué tipo de información guarda su base de fidelización, para no hacer operaciones que no tienen sentido, y quería estimar el ingreso mensual promedio de sus 400 clientes con muestras de solo 20, con pruebas de que la muestra refleja a toda la base. De manera que a continuación presento cómo revisé cada variable y cómo puse a prueba tres maneras de muestrear, usando Python en Google Colab y R en Posit Cloud.

2. Objetivos

Objetivo general: determinar qué operaciones admite cada variable de la base de fidelización y evaluar qué diseño de muestreo produce una estimación confiable del ingreso mensual promedio.

1. Contrastar el tipo de dato que infieren pandas y R con la clasificación que corresponde según el criterio de continuidad y orden.
2. Declarar en código el orden de las variables ordinales y comprobar que quedó bien declarado.
3. Estimar el ingreso medio con muestras por conveniencia, aleatoria simple y estratificada, y medir su error frente al parámetro de la población.

3. Desarrollo de la práctica

El trabajo se hizo sobre el repositorio del laboratorio, completando los TODO de clasificación variables y de muestreo en sus dos versiones. Las capturas de cada paso muestran el código y la salida en Google Colab (Python) y en Posit Cloud (R). En cada paso describo qué se hizo, con qué herramienta y cómo interpreté el resultado.

3.1 Parte A · Clasificación de las variables

A.1 Primera mirada al archivo: lo que dice el programa

El primer paso fue leer data/clientes.csv y pedirle al programa que describiera las columnas. El archivo tiene 12 clientes y 8 columnas. En Python la instrucción clientes.dtypes devolvió int64 para cuatro columnas y object para las otras cuatro, y en R la función str() devolvió la misma división con int y chr. Luego completé el bucle del TODO 2, que recorre cada columna e imprime cuántos valores distintos tiene y cuáles son.



```
[ ]
import pandas as pd

clientes = pd.read_csv("data/clientes.csv")

# ------------------------------------------------------------------------
# 1. Estructura del archivo
# ------------------------------------------------------------------------
print(f"Dimensiones: {clientes.shape[0]} filas x {clientes.shape[1]} columnas")
print()
print("Primeras filas:")
print(clientes.to_string(index=False))
print()

# ------------------------------------------------------------------------
# 2. Tipos inferidos (dtype) — la pista que da el lenguaje
# ------------------------------------------------------------------------
print("Tipos inferidos (dtype):")
print(clientes.dtypes)
print()
# TODO 1: ¿qué columnas llegan como número y cuáles como texto? Anótalo para el
#         informe. Ojo con satisfaccion: llega como número y es ordinal
#         ("4 no es el doble de 2") — la justificación va en el informe.

# ------------------------------------------------------------------------
# 3. Valores únicos — ¿cuántas categorías hay y cuáles son?
# ------------------------------------------------------------------------
# TODO 2: completa el bucle para imprimir, por columna, cuántos valores únicos
#         hay y cuáles son. Pista: for columna in clientes.columns: ...  con
#         clientes[columna].unique() y len(). ¿Cuántas categorías tiene genero?
# for columna in clientes.columns:
#     ...
for columna in clientes.columns:
    u = clientes[columna].unique()
    print(f"{columna}: {len(u)} únicos -> {list(u)}")


# ------------------------------------------------------------------------
# 4. Declarar el orden de las variables ordinales
# ------------------------------------------------------------------------
# Una ordinal se guarda como texto (o como número) y su orden natural se pierde:
# alfabéticamente "Posgrado" iría antes que "Primaria", que no tiene sentido.
#
# TODO 3: convierte nivel_educacion a categoría ordenada.
#         Pista: astype(pd.CategoricalDtype(categories=[...], ordered=True)).
```

Figura 1. *Celda de Colab con la lectura de clientes.csv, la impresión de dtypes y el bucle de valores únicos.*

Con esa salida armé una primera tabla para comparar lo que «cree» el programa con lo que realmente es cada variable. La última columna es la clave del ejercicio: en dos casos el tipo de dato no coincide con la naturaleza de la variable.

Tabla 1. Tipo inferido por el programa frente a la naturaleza de la variable

Variable	Python	R	Valores únicos	¿El tipo inferido basta?
cliente_id	object	chr	12 (uno por fila)	Sí, aunque es un identificador, no un dato para analizar
edad	int64	int	12 (22 a 48)	No del todo: llega entera, pero mide un tiempo continuo
numero_hijos	int64	int	5 (0 a 4)	Sí: es un conteo

Variable	Python	R	Valores únicos	¿El tipo inferido basta?
genero	object	chr	3	Sí, pero hay que notar que no es binaria
nivel_educacion	object	chr	4	No: como texto pierde su orden
ciudad	object	chr	3	Sí
ingresos_mensuales	int64	int	12 (760 a 2450)	Sí: es un monto
satisfaccion	int64	int	4 (2 a 5)	No: parece número, pero es una escala ordenada

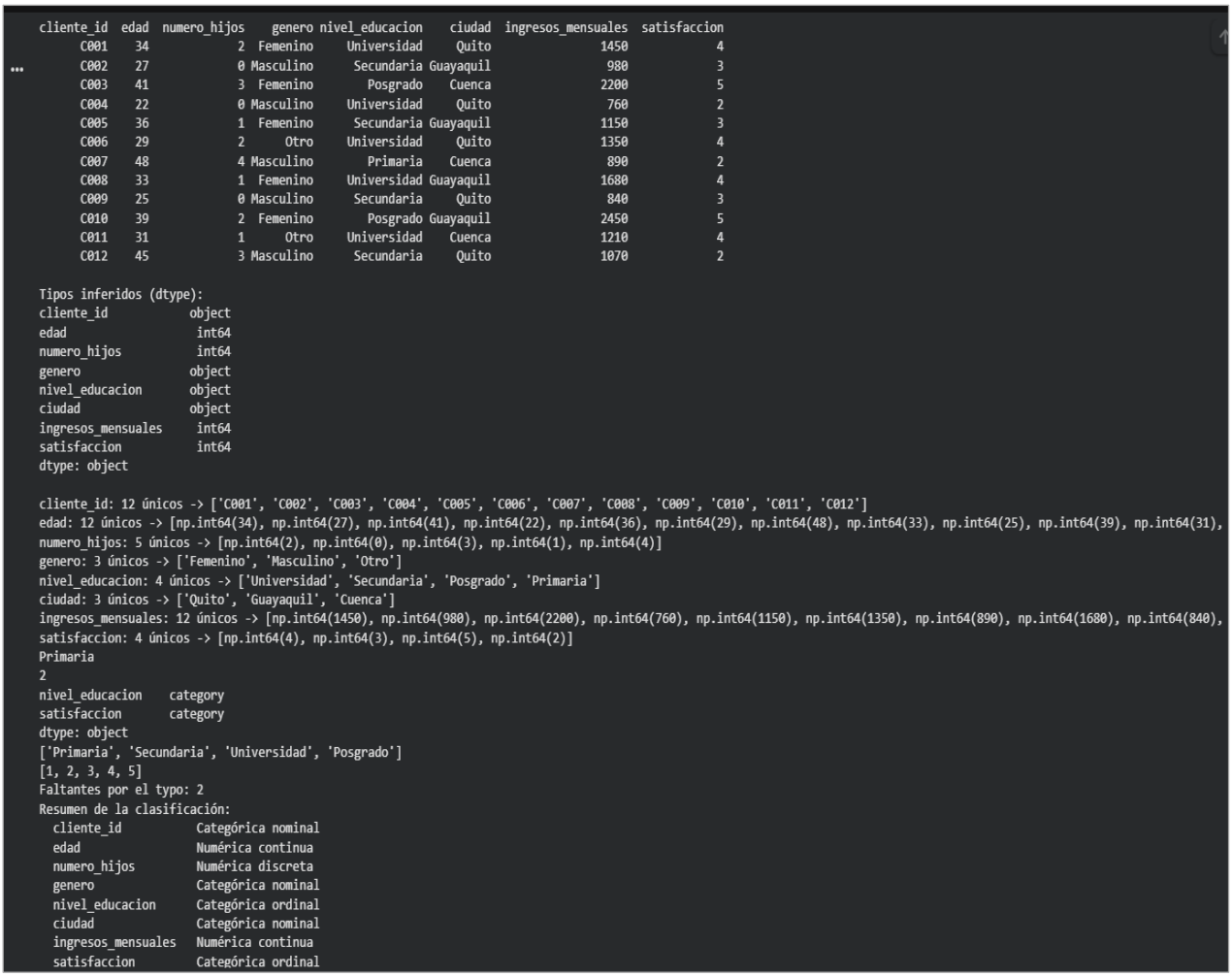


Figura 2. *Salida completa de la Parte A en Colab: datos, tipos inferidos, valores únicos, verificación del orden, prueba del typo y resumen.*

En la salida también apareció algo que no esperaba: los valores numéricos se imprimen como np.int64(34) en lugar de 34. Investigué y se debe a que las versiones recientes de NumPy muestran así sus enteros; no cambia el conteo de valores únicos, solo la forma de mostrarlo.

A.2 Declarar el orden de las ordinales

Para que el programa respete el orden de nivel_educacion y satisfaccion hubo que declararlo de forma explícita. Las instrucciones clave fueron las siguientes:

Tabla 2. Instrucciones usadas para declarar el orden

Lenguaje	Instrucción clave	Qué hace
Python	pd.CategoricalDtype(categories=["Primaria", "Secundaria", "Universidad", "Posgrado"], ordered=True)	Crea un tipo categórico con los niveles en el orden correcto.
Python	clientes["satisfaccion"].astype(pd.CategoricalDtype(categories=[1, 2, 3, 4, 5], ordered=True))	Convierte la escala en categoría ordenada e incluye el nivel 1 aunque nadie lo marcó.
R	factor(clientes$nivel_educacion, levels = c(...), ordered = TRUE)	Crea un factor ordenado con los cuatro niveles.
Ambos	min() y isna().sum() / sum(is.na())	Comprueban el orden y cuentan los datos que se perdieron en la conversión.

```python
# ------------------------------------------------------------------------
# Una ordinal se guarda como texto (o como número) y su orden natural se pierde:
# alfabéticamente "Posgrado" iría antes que "Primaria", que no tiene sentido.
#
# TODO 3: convierte nivel_educacion a categoría ordenada.
#         Pista: astype(pd.CategoricalDtype(categories=[...], ordered=True)).
#         Comprueba con .min(): debe devolver "Primaria".
orden_edu = pd.CategoricalDtype(
    categories=["Primaria", "Secundaria", "Universidad", "Posgrado"], ordered=True)
clientes["nivel_educacion"] = clientes["nivel_educacion"].astype(orden_edu)
print(clientes["nivel_educacion"].min())
# TODO 4: haz lo mismo con satisfaccion (escala 1-5 como etiquetas ordenadas).
#         Prueba extra: ¿qué pasa si en la lista de categorías escribes
#         "Postgrado" (con t) para nivel_educacion? Cuenta los faltantes con
#         .isna().sum() — la trampa del typo es silenciosa.
orden_sat = pd.CategoricalDtype(categories=[1, 2, 3, 4, 5], ordered=True)
clientes["satisfaccion"] = clientes["satisfaccion"].astype(orden_sat)
print(clientes["satisfaccion"].min())

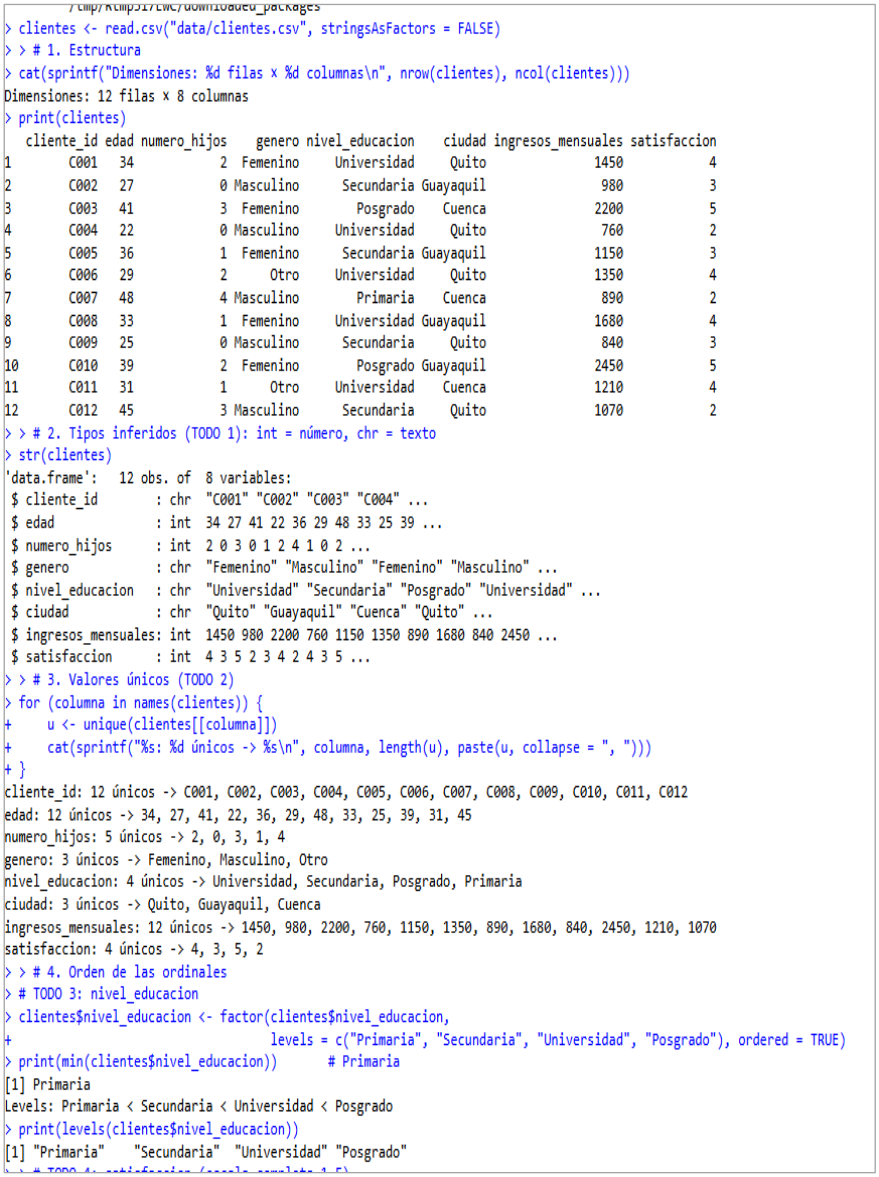
# Prueba extra: el typo Postgrado
typo = pd.CategoricalDtype(
    categories=["Primaria", "Secundaria", "Universidad", "Postgrado"], ordered=True)
prueba = pd.read_csv("data/clientes.csv")["nivel_educacion"].astype(typo)
print("Faltantes por el typo:", prueba.isna().sum())

# ------------------------------------------------------------------------
# 5. Resumen de la clasificación (para el informe)
# ------------------------------------------------------------------------
# TODO 5: completa el resumen con el tipo de cada variable (el criterio que
#         justifica cada clasificación va en el informe).
clasificacion = {
    "cliente_id": "Categórica nominal",
    "edad": "Numérica continua",
    "numero_hijos": "Numérica discreta",
    "genero": "Categórica nominal",
    "nivel_educacion": "Categórica ordinal",
    "ciudad": "Categórica nominal",
    "ingresos_mensuales": "Numérica continua",
    "satisfaccion": "Categórica ordinal",
}
print("Resumen de la clasificación:")
for variable, tipo in clasificacion.items():
    print(f"  {variable:20s} {tipo}")
```

Figura 3. *Colab: declaración del orden con CategoricalDtype, prueba con «Postgrado» mal escrito y diccionario con la clasificación final.*

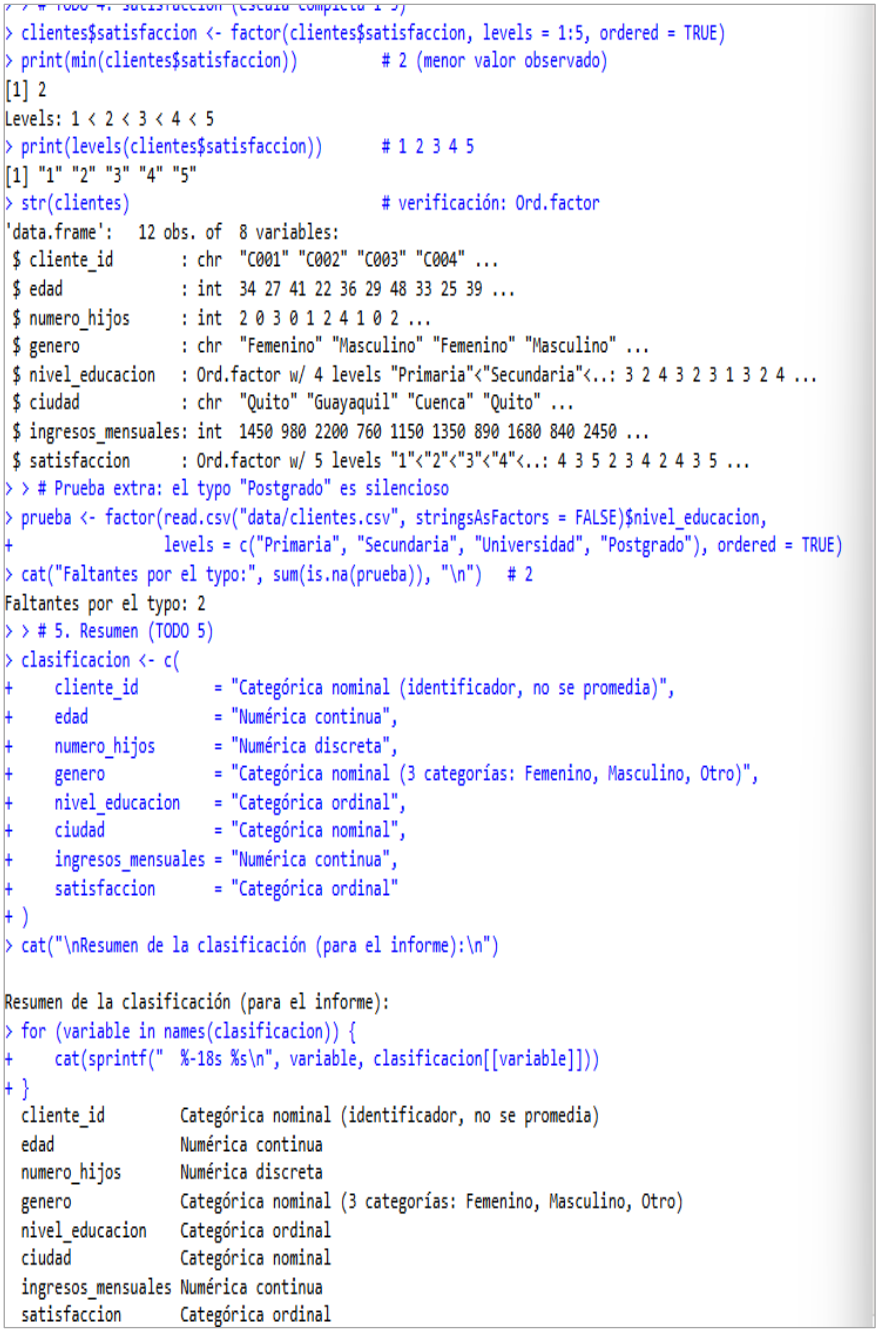
La comprobación funcionó en los dos lenguajes: min() devolvió Primaria para la educación y 2 para la satisfacción, y R mostró Levels: Primaria < Secundaria < Universidad < Posgrado. Lo más

llamativo fue la prueba del error de escritura. Al escribir «Postgrado» en lugar de «Posgrado», los dos clientes con ese nivel (C003 y C010) quedaron como datos faltantes, y ni pandas ni R dieron ningún aviso. Si no se cuentan los faltantes después de convertir, ese error pasaría desapercibido y el análisis trabajaría con 10 personas creyendo que son 12.



```
> clientes <- read.csv("data/clientes.csv", stringsAsFactors = FALSE)
> > # 1. Estructura
> cat(sprintf("Dimensiones: %d filas x %d columnas\n", nrow(clientes), ncol(clientes)))
Dimensiones: 12 filas x 8 columnas
> print(clientes)
   cliente_id edad numero_hijos    genero nivel_educacion    ciudad ingresos_mensuales satisfaccion
1        C001   34            2  Femenino     Universidad     Quito              1450            4
2        C002   27            0 Masculino      Secundaria Guayaquil               980            3
3        C003   41            3  Femenino        Posgrado    Cuenca              2200            5
4        C004   22            0 Masculino     Universidad     Quito               760            2
5        C005   36            1  Femenino      Secundaria Guayaquil              1150            3
6        C006   29            2      Otro     Universidad     Quito              1350            4
7        C007   48            4 Masculino        Primaria    Cuenca               890            2
8        C008   33            1  Femenino     Universidad Guayaquil              1680            4
9        C009   25            0 Masculino      Secundaria     Quito               840            3
10       C010   39            2  Femenino        Posgrado Guayaquil              2450            5
11       C011   31            1      Otro     Universidad    Cuenca              1210            4
12       C012   45            3 Masculino      Secundaria     Quito              1070            2
> > # 2. Tipos inferidos (TODO 1): int = número, chr = texto
> str(clientes)
'data.frame':   12 obs. of  8 variables:
 $ cliente_id       : chr  "C001" "C002" "C003" "C004" ...
 $ edad             : int  34 27 41 22 36 29 48 33 25 39 ...
 $ numero_hijos     : int  2 0 3 0 1 2 4 1 0 2 ...
 $ genero           : chr  "Femenino" "Masculino" "Femenino" "Masculino" ...
 $ nivel_educacion  : chr  "Universidad" "Secundaria" "Posgrado" "Universidad" ...
 $ ciudad           : chr  "Quito" "Guayaquil" "Cuenca" "Quito" ...
 $ ingresos_mensuales: int  1450 980 2200 760 1150 1350 890 1680 840 2450 ...
 $ satisfaccion     : int  4 3 5 2 3 4 2 4 3 5 ...
> > # 3. Valores únicos (TODO 2)
> for (columna in names(clientes)) {
+     u <- unique(clientes[[columna]])
+     cat(sprintf("%s: %d únicos -> %s\n", columna, length(u), paste(u, collapse = ", ")))
+ }
cliente_id: 12 únicos -> C001, C002, C003, C004, C005, C006, C007, C008, C009, C010, C011, C012
edad: 12 únicos -> 34, 27, 41, 22, 36, 29, 48, 33, 25, 39, 31, 45
numero_hijos: 5 únicos -> 2, 0, 3, 1, 4
genero: 3 únicos -> Femenino, Masculino, Otro
nivel_educacion: 4 únicos -> Universidad, Secundaria, Posgrado, Primaria
ciudad: 3 únicos -> Quito, Guayaquil, Cuenca
ingresos_mensuales: 12 únicos -> 1450, 980, 2200, 760, 1150, 1350, 890, 1680, 840, 2450, 1210, 1070
satisfaccion: 4 únicos -> 4, 3, 5, 2
> > # 4. Orden de las ordinales
> # TODO 3: nivel_educacion
> clientes$nivel_educacion <- factor(clientes$nivel_educacion,
+                                    levels = c("Primaria", "Secundaria", "Universidad", "Posgrado"), ordered = TRUE)
> print(min(clientes$nivel_educacion))      # Primaria
[1] Primaria
Levels: Primaria < Secundaria < Universidad < Posgrado
> print(levels(clientes$nivel_educacion))
[1] "Primaria"    "Secundaria"  "Universidad" "Posgrado"
```

Figura 4. *Posit Cloud: str() de clientes.csv, valores únicos por columna y factor ordenado de nivel_educacion.*



```
> clientes$satisfaccion <- factor(clientes$satisfaccion, levels = 1:5, ordered = TRUE)
> print(min(clientes$satisfaccion))         # 2 (menor valor observado)
[1] 2
Levels: 1 < 2 < 3 < 4 < 5
> print(levels(clientes$satisfaccion))      # 1 2 3 4 5
[1] "1" "2" "3" "4" "5"
> str(clientes)                             # verificación: Ord.factor
'data.frame':   12 obs. of  8 variables:
 $ cliente_id       : chr  "C001" "C002" "C003" "C004" ...
 $ edad             : int  34 27 41 22 36 29 48 33 25 39 ...
 $ numero_hijos     : int  2 0 3 0 1 2 4 1 0 2 ...
 $ genero           : chr  "Femenino" "Masculino" "Femenino" "Masculino" ...
 $ nivel_educacion  : Ord.factor w/ 4 levels "Primaria"<"Secundaria"<..: 3 2 4 3 2 3 1 3 2 4 ...
 $ ciudad           : chr  "Quito" "Guayaquil" "Cuenca" "Quito" ...
 $ ingresos_mensuales: int  1450 980 2200 760 1150 1350 890 1680 840 2450 ...
 $ satisfaccion     : Ord.factor w/ 5 levels "1"<"2"<"3"<"4"<..: 4 3 5 2 3 4 2 4 3 5 ...
> > # Prueba extra: el typo "Postgrado" es silencioso
> prueba <- factor(read.csv("data/clientes.csv", stringsAsFactors = FALSE)$nivel_educacion,
+                  levels = c("Primaria", "Secundaria", "Universidad", "Postgrado"), ordered = TRUE)
> cat("Faltantes por el typo:", sum(is.na(prueba)), "\n")   # 2
Faltantes por el typo: 2
> > # 5. Resumen (TODO 5)
> clasificacion <- c(
+     cliente_id        = "Categórica nominal (identificador, no se promedia)",
+     edad              = "Numérica continua",
+     numero_hijos      = "Numérica discreta",
+     genero            = "Categórica nominal (3 categorías: Femenino, Masculino, Otro)",
+     nivel_educacion   = "Categórica ordinal",
+     ciudad            = "Categórica nominal",
+     ingresos_mensuales = "Numérica continua",
+     satisfaccion      = "Categórica ordinal"
+ )
> cat("\nResumen de la clasificación (para el informe):\n")

Resumen de la clasificación (para el informe):
> for (variable in names(clasificacion)) {
+     cat(sprintf("  %-18s %s\n", variable, clasificacion[[variable]]))
+ }
  cliente_id        Categórica nominal (identificador, no se promedia)
  edad              Numérica continua
  numero_hijos      Numérica discreta
  genero            Categórica nominal (3 categorías: Femenino, Masculino, Otro)
  nivel_educacion   Categórica ordinal
  ciudad            Categórica nominal
  ingresos_mensuales Numérica continua
  satisfaccion      Categórica ordinal
```

Figura 5. *Posit Cloud: satisfaccion como factor ordenado de 1 a 5, verificación con str() (Ord.factor), 2 NA por el typo y resumen.*

A.3 Clasificación final

Para clasificar me hice dos preguntas sobre cada variable: ¿tiene sentido hablar de la mitad de un valor? y, si es una categoría, ¿se puede decir que un nivel está «por encima» de otro?

Tabla 3. Clasificación de las variables de clientes.csv

Variable	Tipo	Justificación (criterio)	Verificación en código
cliente_id	Categórica nominal	Sirve solo para identificar. «C005» no es mayor que «C004» en ningún sentido útil y no se puede operar.	object/chr; 12 valores distintos en 12 filas.

Variable	Tipo	Justificación (criterio)	Verificación en código
edad	Numérica continua	Entre 33 y 34 años hay infinitos valores posibles; se anota en años cumplidos solo por costumbre de registro.	int64/int; 12 valores (22 a 48).
numero_hijos	Numérica discreta	Sale de contar y salta de entero en entero: no hay 1,5 hijos.	int64/int; 5 valores (0 a 4).
genero	Categórica nominal	Sus tres grupos no tienen jerarquía; con tres categorías deja de ser binaria.	object/chr; Femenino, Masculino, Otro.
nivel_educacion	Categórica ordinal	Hay jerarquía clara, pero no se puede decir que de Primaria a Secundaria haya «la misma distancia» que de Universidad a Posgrado.	Orden declarado; min() = Primaria; Ord.factor con 4 niveles.
ciudad	Categórica nominal	Quito, Guayaquil y Cuenca son lugares, no niveles.	object/chr; 3 valores.
ingresos_mensuales	Numérica continua	Es dinero: 1450,50 USD tiene sentido y el promedio se interpreta.	int64/int; 12 valores (760 a 2450).
satisfaccion	Categórica ordinal	Es una opinión en escala: un 4 indica más satisfacción que un 2, pero no el doble.	int64/int; 4 valores (2 a 5); declarada 1–5, min() = 2.

Con esta clasificación se responde la primera consulta de la gerencia sobre qué se puede hacer con cada columna:

Tabla 4. Operaciones válidas según el tipo de variable

Tipo	Variables	¿Promediar?	¿Ordenar?	¿Contar?
Numérica continua	edad, ingresos_mensuales	Sí	Sí	Sí
Numérica discreta	numero_hijos	Sí	Sí	Sí
Categórica ordinal	nivel_educacion, satisfaccion	No	Sí	Sí
Categórica nominal	cliente_id, genero, ciudad	No	No	Sí

P1. ¿Qué te dijo el dtype de satisfaccion y por qué no basta para clasificarla? ¿Qué pasa si una ordinal queda como texto sin declarar el orden?

El programa dijo que satisfaccion es un entero (int64 en Python, int en R), y en ese sentido no se equivoca: los valores sí están guardados como números. El problema es que el tipo de dato describe el almacenamiento, no el significado. Una calificación de 1 a 5 es una opinión ordenada; nadie puede afirmar que quien marcó 4 está el doble de satisfecho que quien marcó 2. El caso contrario ocurre con una ordinal guardada como texto, como nivel educación donde el programa la ordenaría alfabéticamente, y Posgrado quedaría como el nivel más bajo. Por eso declaré el orden y lo verifiqué con min(), que devolvió 2 y Primaria. También declaré el nivel 1 de la satisfacción aunque nadie lo usó, para no perder la escala completa.

3.2 Parte B · Población vs. muestra

B.1 La población y su parámetro

La base completa data/clientes_400.csv hace el papel de población. Al cargarla, el script confirmó 400 clientes repartidos en Quito (200), Guayaquil (120) y Cuenca (80), y calculó el promedio real de ingresos_mensuales, que fue el mismo en Python y en R.

Tabla 5. La población y su parámetro

Concepto	Valor
Población (N)	400 clientes de la base de fidelización
Variable de interés	ingresos_mensuales (USD)
Parámetro poblacional (media)	1330.72 USD
¿Por qué se muestrea si aquí el parámetro existe y se calculó?	Porque este caso es un ensayo controlado. En la vida real la empresa no tiene el ingreso de todos sus clientes y tendría que encuestarlos, lo que cuesta tiempo y dinero. Conocer el valor real permite medir cuánto se equivoca cada diseño antes de usarlo en un estudio de verdad.

```
[ ]
# muestreo.py — Parte B: población vs. muestra (RetailAndina).
#
# La población es la base completa de fidelización: data/clientes_400.csv (N = 400).
# La pregunta de negocio: ¿cuál es el ingreso mensual promedio de los clientes?
# Se estima con tres diseños de muestreo y se compara cada estimación con el
# parámetro real, que aquí sí se conoce porque la población es pequeña y visible.
#
# Uso (desde la raíz del repositorio):  python scripts/muestreo.py

import pandas as pd

pd.set_option("display.width", 100)


def reporte(nombre, muestra, parametro):
    """Imprime n, media estimada y composición por ciudad de una muestra.

    Composición por ciudad: es la pista que delata un sesgo de selección.
    """
    if muestra is None:
        print(f"[{nombre}] pendiente de completar (TODO)")
        return
    n = len(muestra)
    media = muestra["ingresos_mensuales"].mean()
    comp = muestra["ciudad"].value_counts().to_dict()
    print(f"[{nombre}] n={n}  media={media:.2f} USD  "
          f"(parámetro {parametro:.2f}, diferencia {media - parametro:+.2f})")
    print(f"    composición por ciudad: {comp}")


# ---------------------------------------------------------------------------
# 1. La población y su parámetro (N = 400)
# ---------------------------------------------------------------------------
# La base de fidelización completa es la población: 400 clientes. El parámetro
# es el ingreso mensual promedio real (solo calculable porque la población es
# pequeña y accesible: sirve de espejo para juzgar las muestras).
poblacion = pd.read_csv("data/clientes_400.csv")
parametro = poblacion["ingresos_mensuales"].mean()
print(f"Población: {poblacion.shape[0]} clientes x {poblacion.shape[1]} columnas")
print(f"Parámetro poblacional — ingreso medio: {parametro:.2f} USD")
print("Ciudades en la población:", poblacion["ciudad"].value_counts().to_dict())
print()

# ---------------------------------------------------------------------------
```

Figura 6. *Colab: función reporte, lectura de clientes_400.csv y cálculo del parámetro poblacional.*

B.2 Los tres diseños de muestreo

Cada diseño toma 20 clientes, es decir, el 5 % de la base. Lo que cambia es la forma de elegirlos. En la siguiente tabla resumo la instrucción usada en cada lenguaje:

Tabla 6. Instrucción clave de cada diseño

Diseño	Python (pandas)	R	Cómo elige a los clientes
Conveniencia	poblacion.head(20)	head(poblacion, 20)	Toma las primeras 20 filas, sin sorteo.
Aleatorio simple	poblacion.sample(20, random_state=1)	set.seed(1) + sample(nrow(poblacion), 20)	Sortea 20; todos tienen la misma probabilidad.
Estratificado	groupby("ciudad").sample(frac=0.05)	group_by(ciudad) |> slice_sample(prop = 0.05)	Sortea el 5 % dentro de cada ciudad.

```
muestra_conv = poblacion.head(20)


# ------------------------------------------------------------------------
# 3. Muestra aleatoria simple: el sorteo de 20
# ------------------------------------------------------------------------
# TODO 2: sortea 20 clientes con la misma probabilidad para todos (20/400 = 5 %).
#         Pista: .sample(20, random_state=1). Con random_state el resultado es
#         reproducible; sin él cambia en cada ejecución.
# TODO 3: repite el sorteo con las semillas 2 a 5 y anota las 5 medias en el
#         informe: la variabilidad de la estimación es parte de la lección.
muestra_aas = poblacion.sample(20,random_state=1)
medias = {}
for semilla in range(1, 6):
    m = poblacion.sample(20, random_state=semilla)
    medias[semilla] = round(m["ingresos_mensuales"].mean(), 2)
print("Medias por semilla:", medias)

# ------------------------------------------------------------------------
# 4. Muestra estratificada: 5 % dentro de cada ciudad
# ------------------------------------------------------------------------
# TODO 4: muestrea dentro de cada ciudad (el estrato), con afijación
#         proporcional: 5 % de cada una → 10 de Quito, 6 de Guayaquil y 4 de
#         Cuenca (20 en total).
#         Pista: .groupby("ciudad") y .sample(frac=0.05, random_state=1).
#         ¿Qué garantiza este diseño que el sorteo simple no garantiza?
muestra_est = poblacion.groupby("ciudad").sample(frac=0.05, random_state=1)

# ------------------------------------------------------------------------
# 5. Comparación final
# ------------------------------------------------------------------------
reporte("Conveniencia", muestra_conv, parametro)
reporte("Aleatorio simple", muestra_aas, parametro)
reporte("Estratificado", muestra_est, parametro)
print()

if muestra_conv is not None and muestra_aas is not None and muestra_est is not None:
    resumen = pd.DataFrame([
        {"diseño": "Conveniencia (head)", "n": len(muestra_conv),
         "media estimada": round(muestra_conv["ingresos_mensuales"].mean(), 2)},
        {"diseño": "Aleatorio simple (sample)", "n": len(muestra_aas),
         "media estimada": round(muestra_aas["ingresos_mensuales"].mean(), 2)},
        {"diseño": "Estratificado (groupby + frac)", "n": len(muestra_est),
         "media estimada": round(muestra_est["ingresos_mensuales"].mean(), 2)},
```

Figura 7. *Colab: implementación de head(20), sample con semillas 1 a 5, groupby con frac = 0.05 y comparación final.*

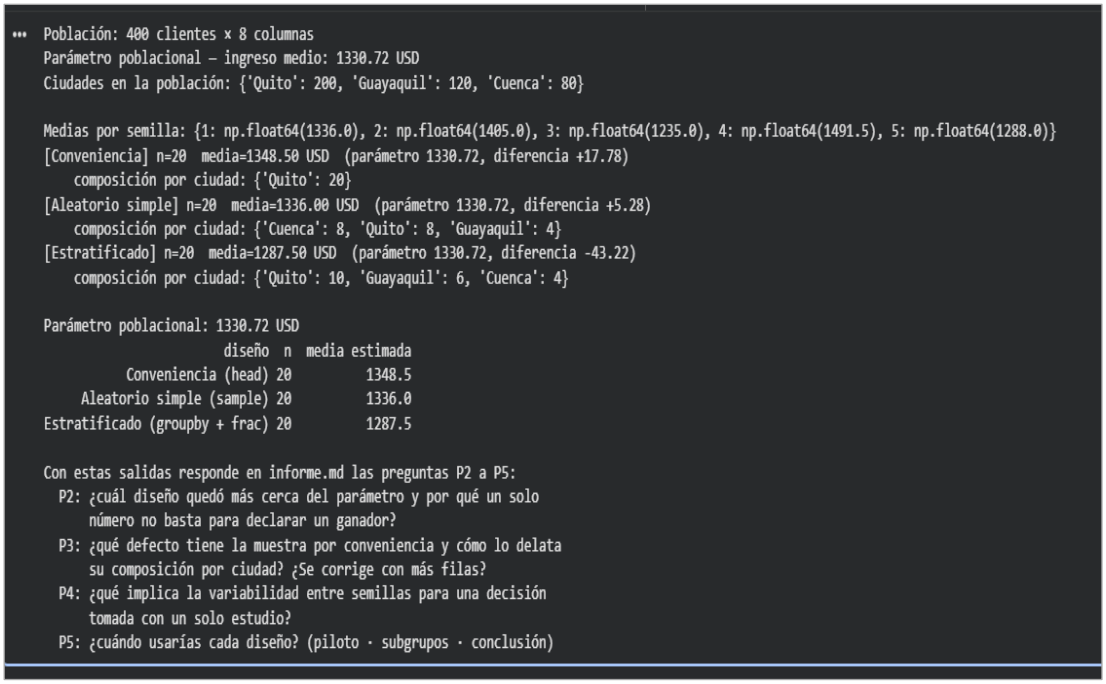


Figura 8. *Salida de la Parte B en Colab: parámetro, medias por semilla y resultado de cada diseño con su composición por ciudad.*

Los resultados de Python con la semilla 1 se resumen en la Tabla 7. Agregué el error relativo para ver la diferencia en proporción al valor real.

Tabla 7. Resultados de los tres diseños (Python, semilla 1)

Diseño	n	Media estimada	Composición por ciudad	Diferencia vs. parámetro	Error relativo
Conveniencia (head)	20	1348.50	Quito 20	+17.78	+1.34 %
Aleatorio simple (sample)	20	1336.00	Cuenca 8, Quito 8, Guayaquil 4	+5.28	+0.40 %
Estratificado (groupby + 5 %)	20	1287.50	Quito 10, Guayaquil 6, Cuenca 4	−43.22	−3.25 %

B.3 Repetición del sorteo y versión en R

Un solo sorteo no dice mucho, así que repetí el aleatorio simple con las semillas 1 a 5. Hice lo mismo en R, donde las semillas generan muestras distintas porque R y NumPy usan generadores de números aleatorios diferentes. Por eso los valores no coinciden entre lenguajes, aunque el comportamiento es el mismo.

Tabla 8. Medias del aleatorio simple con cinco semillas (USD)

Semilla	1	2	3	4	5	Promedio	Desv. est.
Python	1336.00	1405.00	1235.00	1491.50	1288.00	1351.10	100.38
R	1318.00	1287.50	1233.00	1385.50	1283.00	1301.40	56.03

```
> # La base de fidelización completa es la población: 400 clientes. El parámetro
> # es el ingreso mensual promedio real (solo calculable porque la población es
> # pequeña y accesible: sirve de espejo para juzgar las muestras).
> cat(sprintf("Población: %d clientes x %d columnas\n", nrow(poblacion), ncol(poblacion)))
Población: 400 clientes x 8 columnas
> parametro <- mean(poblacion$ingresos_mensuales)
> cat(sprintf("Parámetro poblacional — ingreso medio: %.2f USD\n", parametro))
Parámetro poblacional — ingreso medio: 1330.72 USD
> cat("Ciudades en la población: "); print(table(poblacion$ciudad))
Ciudades en la población:
   Cuenca Guayaquil     Quito
       80       120       200
>
> # ---------------------------------------------------------------------------
> # 2. Muestra por conveniencia: "los primeros que llegaron"
> # ---------------------------------------------------------------------------
> # TODO 1: toma como muestra las primeras 20 filas de la población.
> #         Pista: head(poblacion, 20) — igual que la demo de clase.
> #         Después de correrlo, mira la composición por ciudad que imprime el
> #         reporte: ¿es un corte al azar? ¿alguna ciudad falta por completo?
> muestra_conv <- head(poblacion, 20)
>
> # ---------------------------------------------------------------------------
> # 3. Muestra aleatoria simple: el sorteo de 20
> # ---------------------------------------------------------------------------
> # TODO 2: sortea 20 clientes con la misma probabilidad para todos (20/400 = 5 %).
> #         Pista: poblacion[sample(nrow(poblacion), 20), ] con set.seed(1)
> #         delante para que sea reproducible (sin set.seed cambia cada vez).
> # TODO 3: repite el sorteo con las semillas 2 a 5 y anota las 5 medias en el
> #         informe: la variabilidad de la estimación es parte de la lección.
> #         Pista: for (semilla in 1:5) { set.seed(semilla); ... }
> set.seed(1)
> muestra_aas <- poblacion[sample(nrow(poblacion), 20), ]
>
> medias_semillas <- numeric(5)
> for (semilla in 1:5) {
+     set.seed(semilla)
+     m <- poblacion[sample(nrow(poblacion), 20), ]
+     medias_semillas[semilla] <- mean(m$ingresos_mensuales)
+ }
> cat("\nMedias del aleatorio simple por semilla:\n")

Medias del aleatorio simple por semilla:
> print(data.frame(semilla = 1:5, media = round(medias_semillas, 2)), row.names = FALSE)
 semilla  media
       1 1318.0
       2 1287.5
       3 1233.0
       4 1385.5
       5 1283.0
> cat(sprintf("Rango de medias: %.2f a %.2f\n\n", min(medias_semillas), max(medias_semillas)))
Rango de medias: 1233.00 a 1385.50
```

Figura 9. *Posit Cloud: población de 400 clientes, parámetro, muestra por conveniencia y medias del aleatorio simple con semillas 1 a 5.*

```
> #         ¿Qué garantiza este diseño que el sorteo simple no garantiza?
> suppressPackageStartupMessages(library(dplyr))
> set.seed(1)
> muestra_est <- poblacion |>
+     group_by(ciudad) |>
+     slice_sample(prop = 0.05) |>
+     ungroup()
>
> # ------------------------------------------------------------------------
> # 5. Comparación final
> # ------------------------------------------------------------------------
> reporte("Conveniencia", muestra_conv, parametro)
[Conveniencia] n=20  media=1348.50 USD  (parámetro 1330.72, diferencia +17.78)
    composición por ciudad:
Quito
   20
> reporte("Aleatorio simple", muestra_aas, parametro)
[Aleatorio simple] n=20  media=1318.00 USD  (parámetro 1330.72, diferencia -12.72)
    composición por ciudad:
   Cuenca Guayaquil     Quito
       5         7         8
> reporte("Estratificado", muestra_est, parametro)
[Estratificado] n=20  media=1226.00 USD  (parámetro 1330.72, diferencia -104.72)
    composición por ciudad:
   Cuenca Guayaquil     Quito
       4         6        10
>
> if (!is.null(muestra_conv) && !is.null(muestra_aas) && !is.null(muestra_est)) {
+     resumen <- data.frame(
+         diseno = c("Conveniencia (head)", "Aleatorio simple (sample)", "Estratificado (slice_sample)"),
+         n = c(nrow(muestra_conv), nrow(muestra_aas), nrow(muestra_est)),
+         media_estimada = round(c(mean(muestra_conv$ingresos_mensuales),
+                                  mean(muestra_aas$ingresos_mensuales),
+                                  mean(muestra_est$ingresos_mensuales)), 2)
+     )
+     cat(sprintf("Parámetro poblacional: %.2f USD\n", parametro))
+     print(resumen, row.names = FALSE)
+     cat("Con estas salidas responde en informe.md las preguntas P2 a P5:\n")
+     cat("  P2: ¿cuál diseño quedó más cerca del parámetro y por qué un solo\n")
+     cat("      número no basta para declarar un ganador?\n")
+     cat("  P3: ¿qué defecto tiene la muestra por conveniencia y cómo lo delata\n")
+     cat("      su composición por ciudad? ¿Se corrige con más filas?\n")
+     cat("  P4: ¿qué implica la variabilidad entre semillas para una decisión\n")
+     cat("      tomada con un solo estudio?\n")
+     cat("  P5: ¿cuándo usarías cada diseño? (piloto · subgrupos · conclusión)\n")
+ } else {
+     cat("Completa los TODO 1 a 4 para ver la tabla comparativa.\n")
+ }
Parámetro poblacional: 1330.72 USD
                      diseno  n media_estimada
         Conveniencia (head) 20         1348.5
    Aleatorio simple (sample) 20         1318.0
 Estratificado (slice_sample) 20         1226.0
```

Figura 10. *Posit Cloud: muestra estratificada con group_by y slice_sample y tabla comparativa de los tres diseños en R.*

En R, la conveniencia volvió a dar 1348.50 USD con 20 clientes de Quito no hay sorteo, así que el resultado no cambia), el aleatorio simple dio 1318.00 USD con Cuenca 5, Guayaquil 7 y Quito 8, y el estratificado 1226.00 USD con exactamente 10, 6 y 4 clientes por ciudad.

B.4 Respuestas a las preguntas

P2. ¿Cuánto vale el parámetro y cuánto estimó cada diseño? ¿Cuál quedó más cerca y por qué eso no alcanza para declarar un ganador?

El parámetro es 1330.72 USD. En Python, el aleatorio simple quedó más cerca 1336.00 USD seguido de la conveniencia 1348.50 USD, y del estratificado 1287.50 USD. Si me quedara solo con ese orden, el estratificado parecería el peor, pero la Tabla 8 muestra que el mismo aleatorio simple puede alejarse hasta 160.78 USD del parámetro con otra semilla. Es decir, el primer lugar del sorteo 1 fue en parte suerte. Además, en R el ranking cambió. Para declarar un ganador habría que repetir cada diseño muchas veces y comparar cuánto se alejan en promedio, no una sola corrida.

P3. La muestra por conveniencia también tiene 20 filas. ¿Qué defecto tiene y cómo lo delatas en la composición por ciudad? ¿Se corrige tomando más filas?

Su defecto es que no representa a la base. La composición por ciudad lo deja en evidencia: 20 de 20 clientes son de Quito, cuando Quito es la mitad de la población, y no aparece nadie de Guayaquil ni de Cuenca. Esto pasa porque el archivo viene agrupado por ciudad y head() simplemente lee desde arriba. Tomar más filas no lo arregla, porque mientras el archivo esté ordenado así, las 40 o 100 primeras filas seguirían siendo de Quito. Una corrección sencilla sería desordenar las filas antes de tomarlas.

P4. El sorteo aleatorio dio un número distinto en cada semilla. ¿Qué implica esa variabilidad para una empresa que va a decidir con un solo estudio?

Con muestras de 20, las medias de Python tuvieron una desviación estándar de 100.38 USD. Eso significa que un estudio único puede quedar fácilmente 100 USD por encima o por debajo del valor real sin que nadie se dé cuenta. Si la gerencia usara ese número para fijar el precio de una tarjeta de crédito de la tienda o el monto de un cupón, podría equivocarse de segmento. La implicación práctica es que el resultado debe acompañarse de un margen de error, y que si se necesita más precisión hay que aumentar el tamaño de la muestra.

P5. ¿Cuándo usarías cada diseño en la práctica?

Usaría la conveniencia para pruebas internas, como comprobar que el script corre o que una encuesta nueva se entiende, sin sacar conclusiones de los números. El aleatorio simple lo usaría para una pregunta general sobre toda la cadena, como el ingreso promedio de todos los clientes, cuando no importa cuántos salgan de cada ciudad. El estratificado lo usaría cuando las ciudades deben compararse o todas deben estar presentes, por ejemplo para decidir cuánto presupuesto de marketing asignar a cada una.

4. ¿Qué aprendí con esta práctica?

Antes de esta práctica pensaba que una muestra más grande siempre era mejor y que el tipo que asigna Python a una columna ya era su clasificación. Ahora sé que ninguna de las dos ideas es correcta. Con satisfaccion aprendí a desconfiar de un int64 y a preguntarme qué representa el número. Con la prueba de «Postgrado» aprendí que convertir datos puede borrar información en silencio, así que ahora reviso los faltantes después de cada conversión.

Lo que más me costó entender fue por qué la muestra por conveniencia, que quedó a menos de 18 USD del valor real, se considera la peor. La respuesta estuvo en mirar la composición y no solo la media: acertó por casualidad, pero su forma de elegir excluye a la mitad de los clientes. También me sorprendió que R y Python dieran muestras distintas con la misma semilla. Si repitiera el laboratorio, haría un ciclo de 1000 sorteos por diseño y graficaría un histograma de las medias, para comparar los diseños con evidencia más sólida que una sola corrida.

5. Conclusiones

1. El tipo de dato que asigna el programa es solo un punto de partida. En esta base, 2 de las 8 columnas edad y satisfaccion quedaron mal descritas por su tipo, y solo el criterio permitió clasificarlas. Declarar el orden fue necesario para que min() diera «Primaria» y 2, y un nivel mal escrito borró 2 de 12 registros sin aviso.

2. La representatividad depende de cómo se elige la muestra y no de cuántas filas tiene. Con el mismo tamaño de 20, head() dejó fuera a Guayaquil y Cuenca, que suman el 50 % de los clientes, mientras que el estratificado respetó siempre la proporción 10/6/4.
3. Toda estimación muestral lleva incertidumbre. Las cinco medias del aleatorio simple variaron con una desviación de 100.38 USD en Python, por lo que ningún diseño puede juzgarse con una sola corrida.

Para estimar el ingreso mensual promedio de la base de 400 clientes valor real de 1330.72 USD se recomienda una muestra aleatoria estratificada por ciudad, reportada con su margen de error, y nunca las primeras filas del archivo.

6. Bibliografía

Cochran, W. G. (1977). *Sampling Techniques* (3.ª ed.). John Wiley & Sons.

VanderPlas, J. (2023). *Python Data Science Handbook* (2.ª ed.). O'Reilly Media.

Wickham, H., Çetinkaya-Rundel, M. y Grolemund, G. (2023). *R for Data Science* (2.ª ed.). O'Reilly Media. https://r4ds.hadley.nz/

pandas development team. (2026). *pandas.core.groupby.DataFrameGroupBy.sample*. https://pandas.pydata.org/docs/reference/api/pandas.core.groupby.DataFrameGroupBy.sample.html

Posit. (2026). *slice: Subset rows using their positions* [Documentación de dplyr]. https://dplyr.tidyverse.org/reference/slice.html

Escuela Politécnica Nacional. (2026). *Laboratorio: clasificación de variables y población vs. muestra* [Repositorio, scripts y datos clientes.csv y clientes_400.csv]. IDSD422, 2026-B.

Declaración de uso de IA: utilicé Claude (Anthropic) para organizar la estructura del informe y revisar la redacción. Las interpretaciones las contrasté con las salidas de los scripts.